

CHAPTER FOUR

SYSTEM IMPLEMENTATION, TESTING, RESULTS AND DISCUSSION

4.1 System Implementation Overview

This chapter presents the implementation, testing, results and discussion of the personality prediction system. It explains how the proposed methods were brought together in the application and examines the results obtained from the PANDORA experiment. The discussion follows the objectives stated in Chapter One and the experimental procedure described in Chapter Three. Attention is given to the preparation of author records, comment selection, contextual feature extraction, data augmentation and prediction of the five Big Five personality traits.

The implemented system is a web-based research platform developed with Django. It provides facilities for starting an experiment, storing its settings, viewing the experimental conditions and examining their results. The machine learning components compare sparse regression with a stacked bidirectional Long Short-Term Memory model. Each model is evaluated with baseline and Q-learning comment selection, with and without Generative Adversarial Network augmentation. These combinations produce eight experimental conditions. They represent eight processing combinations rather than eight folds of cross-validation.

The main results discussed in this chapter are taken from the saved experiment identified as `run_20260917_133820`. This identifier is retained so that the figures and tables can be linked to one set of settings and observations. The experiment contains 50 training authors, 10 validation authors and 10 test authors. The numerical results are therefore findings from a small, single experimental run. They provide evidence about the behaviour of the implemented system, but do not establish that the same performance will be obtained on every sample of social media authors.

The chapter distinguishes between the implementation of a feature and evidence that the feature improves prediction. For example, the successful generation of synthetic samples confirms that the augmentation stage operated. Its usefulness must still be judged from the held-out results. Similarly, fewer selected comments indicate reduced input volume, while an

actual reduction in processing time requires separate timing evidence. This distinction guides the presentation of the findings throughout the chapter.

The final choice of a preferred pipeline is deliberately left open. The results are discussed in terms of prediction error, classification behaviour, comment usage and recorded training time. The researcher will complete the final selection after considering these findings and resolving the reporting issues identified in the classification results. The marked insertion areas indicate the supporting tables, charts, code extracts and screenshots required for the completed report.

4.2 Implementation Environment and System Requirements

4.2.1 Software Components

Python provides the main programming environment for the application and its machine learning services. Django handles the researcher interface, request processing and storage of experiment records. The separation of the application into services allows the data loader, comment selector, BERT encoder, augmentation component and prediction models to perform distinct tasks. This arrangement also makes it possible to inspect a particular stage without repeating the explanation of the entire system.

The contextual representation stage uses the pretrained bert-base-uncased model. Each selected comment is converted into a 768-dimensional representation before author-level features are assembled. The sparse regression branch uses pooled comment representations, while the LSTM branch uses sequences of comment representations. The stored configuration for the reported experiment specifies ElasticNet regularization for the branch whose saved condition names begin with lasso. Consequently, the term sparse regression is used in this chapter where it refers to the actual fitted branch.

The local application uses a relational database for research records and separate artifact folders for fitted models and experimental reports. This division is useful because a database record can identify an experiment without storing every model parameter inside the same table. The artifact folder contains the condition metrics, model states, threshold records and supporting reports needed to inspect the run. The version of the application and the saved configuration must be retained alongside these files when the experiment is reproduced.

[Insert Table 4.1 here: Software and hardware environment. Include component, purpose and verified version or specification. Record the Python, Django, PyTorch, Transformers and scikit-learn versions from the environment that executed the experiment. Add the processor, installed memory, operating system and GPU status from that same computer. These details are not fully recorded in the saved run and should be verified before the table is completed.]

4.2.2 Hardware and Execution Requirements

The experiment requires sufficient memory to load the language model, retain the selected comment representations and train the prediction models. The amount of text differs considerably between authors. A model that accepts the full comment sequence may therefore require more processing time than a model fitted on one pooled vector per author. The stored results reflect this difference, particularly in the augmented LSTM condition that uses the full comment history.

Hardware requirements should be interpreted in relation to the task being performed. Viewing stored results through the web interface is different from generating embeddings or training an augmented sequence model. A computer that displays the dashboard successfully has not necessarily demonstrated adequate resources for the longest training condition. For this reason, the final hardware table should describe the machine used for the experiment rather than present an untested minimum requirement.

The PANDORA experiment is started through a background thread in the current application. This allows the researcher to initiate a run and return to its status page while processing continues. The completion of an experiment still depends on the application process remaining available. The implementation therefore provides a practical research workflow, while the present evidence does not establish behaviour under heavy concurrent use or prolonged server interruption.

4.3 User Interface and Data Storage Implementation

4.3.1 Researcher Experiment Interface

The researcher interface connects the experimental settings to the machine learning workflow. A researcher can specify the required sample and training settings before starting a

PANDORA experiment. The application associates the experiment with the researcher and stores information about its status and artifact location. These records make it possible to distinguish one experiment from another when several runs have been carried out.

The run details and results pages support the inspection of the different conditions. A condition identifies the model family, comment-selection approach and augmentation setting used for that result. This is necessary because a single experiment produces several fitted pipelines. Presenting them under a common run helps the researcher compare conditions evaluated on the same author partitions instead of accidentally combining results from unrelated runs.

[Insert Figure 4.1 here: Researcher experiment setup and completed-run interface. Use two clearly labelled screenshots from the application, showing the settings and the eight condition results for the same run. Retain readable labels and the run identifier. Remove personal account details and avoid using the public demonstration page as evidence of experimental performance.]

The interface provides a means of reviewing the recorded results, but the interpretation should also refer to the saved reports. A dashboard value is only meaningful when its metric, dataset split and averaging method are understood. During the review of the current run, the classification fields were found to require further alignment between model families. This issue is discussed in Section 4.8 so that the presentation does not imply a comparison that the saved fields cannot support.

4.3.2 Experiment Records and Prediction Profiles

The implemented data structure separates the experiment run from its condition results, threshold results and dataset allocations. The experiment run identifies the shared settings. A condition result identifies one processing combination and its stored measures. Threshold records identify the split, trait, condition and candidate threshold associated with the classification measures. Dataset allocations record the authors assigned to the experiment and support the inspection of membership and reuse.

The application also provides prediction-run and test-profile records for later prediction batches. A prediction run identifies the batch and its selected condition, while individual profiles retain observed and predicted trait values where these are available. The

separation between a batch and an individual profile avoids treating a group summary as though it described every author. It also supports the inspection of cases where the prediction differs from the observed score.

The database relationships described in Chapter Three are represented in the application models. However, database storage alone does not establish the predictive quality of a model. Its contribution is to preserve the connection between an experiment, its settings and its outputs. The interpretation of those outputs still depends on correct data separation and consistent evaluation. These requirements apply even when the records have been saved successfully.

4.3.3 Scope of the Prediction Display

The application contains features developed at different stages of the project. The current PANDORA experiment is the basis of the results in this chapter. Earlier participant-management and questionnaire-related features may remain available, but they are not the source of the 70-author experimental sample. The source labels in this run come from the supplied PANDORA files rather than a new questionnaire administered through the application.

The public demonstration interface is also separate from the experimental evidence. Its heuristic output should not be used to claim the performance of a trained PANDORA model. In addition, later prediction batches must follow the same preparation and comment-selection procedure as their source condition before they can be treated as equivalent applications of that pipeline. The current batch embedding helper takes an initial subset of comments, so a profile screenshot by itself does not demonstrate full reproduction of the experimental selection procedure.

[Insert Figure 4.2 here: Stored prediction batch and individual OCEAN profile. Use a genuine saved profile, hide the author identifier and state the source run and condition in the caption. Present this screenshot as evidence of the implemented output display. Only describe it as a faithful application of the experimental pipeline after the comment preparation and selection settings have been checked.]

4.4 Dataset Preparation and Experimental Sample

4.4.1 Author-Level Data Separation

The reported experiment uses the file-defined training, validation and test partitions. The saved data report records 50 training authors, 10 validation authors and 10 test authors, giving a total of 70 authors. The author is the unit of prediction. Each author is represented by a collection of comments and five observed personality scores. The number of comments should therefore not be confused with the number of independent labelled author observations.

The saved overlap report contains zero shared author identifiers between training and validation, training and test, and validation and test. This supports the intended author-level separation for the sampled records. The use of a shared author sample across the eight conditions allows matched comparisons: a change in model performance can be examined without changing the authors being evaluated. This is different from reusing a training author as a held-out test author.

The training records support the fitting of the selector, prediction models and augmentation component. Validation records support model decisions, including the selection of decision thresholds and the retention of the LSTM training state. Test records support the final held-out measurements. The reported absence of identifier overlap is useful evidence, although it does not by itself prove that every source-text duplication or possible data-quality problem has been excluded.

[Insert Table 4.2 here: Experimental sample and comment distribution. Use experiment_data.json and data_quality.json from the reported run. Columns should show split, authors, comments, mean comments per author, median comments per author, minimum and maximum. The author counts are 50, 10 and 10; the comment counts are 8,675, 681 and 1,663 for training, validation and test respectively. Add a note stating that all three pairwise author-overlap counts are zero.]

4.4.2 Comment Volume and Retention

The sampled records contain 11,019 comments. Training authors account for 8,675 comments, validation authors for 681 comments and test authors for 1,663 comments. The corresponding mean comment counts are 173.5, 68.1 and 166.3. However, the medians are

77.5, 57.0 and 23.5 respectively. The difference between the mean and median, especially in the test set, indicates that a small number of authors contribute much longer histories than others.

This variation matters for both prediction and computation. An author with a long history provides many more comment embeddings to the LSTM than an author with a short history. Pooling reduces these histories to vectors of equal dimension, while sequence processing retains their unequal lengths. The resulting comparison therefore concerns two different ways of representing the same author information, rather than simply two algorithms receiving identical input shapes.

The run records that the file-defined datasets were loaded as already cleaned. It reports 11,019 comments before and after the experiment's cleaning stage and no additional exclusions for duplicate, short or invalid comments inside that stage. All 70 sampled authors were retained. These figures describe retention within this experiment. They should not be presented as evidence that no comments were removed during earlier dataset preparation or that the original PANDORA collection contained no unsuitable records.

4.4.3 Personality Scores and Label Boundaries

The supplied personality scores are represented on a normalized scale by dividing the source values by 100. The primary prediction task remains estimation of these continuous OCEAN scores. For secondary Low/High evaluation, the observed scores are converted into binary labels using trait boundaries calculated from the training authors. The recorded training medians are 0.790 for Openness, 0.355 for Conscientiousness, 0.250 for Extraversion, 0.350 for Agreeableness and 0.460 for Neuroticism.

These observed-label boundaries serve a different purpose from the model decision thresholds. The training median defines which observed scores count as Low or High. A validation-selected threshold determines how the model's continuous prediction is converted into a predicted Low or High label. Keeping the two separate is necessary when interpreting both the tables and the graphs. A model decision threshold should not be described as the median merely because the observed labels use training medians.

The median-based labels are relative to the training sample and are used for this experiment's evaluation. They are not clinical categories or fixed population standards. They

also do not guarantee equal numbers of Low and High authors in validation or test data. The distribution of these held-out labels should therefore accompany classification results, particularly because each held-out set contains only ten authors.

4.5 Implementation of the Eight Experimental Conditions

4.5.1 Baseline and Q-Learning Comment Selection

The baseline condition uses the full available comment history for each sampled author. The Q-learning condition learns a select-or-skip policy using the training authors and then applies the learned selection procedure to the author profiles. The recorded policy does not impose a fixed number of ten comments. Although a `top_k` setting remains in the configuration, the current selection report explicitly records no fixed selection budget. The findings must therefore describe the actual selection behaviour rather than assume that every author contributed the same number of comments.

The purpose of Q-learning in this project is to reduce the amount of text passed to the BERT encoder while retaining useful information. Its reward uses text-based measures of informativeness and redundancy together with a selection cost. It is not directly rewarded by improvement in personality prediction error. The use of such a reward follows the general action-and-reward principle of reinforcement learning, but the practical value of this particular policy must be assessed from the experiment (Sutton & Barto, 2018).

The recorded Q-learning settings use three training epochs and random seed 42. The same selection setting is shared by the sparse and sequence branches within the relevant conditions. This arrangement allows the effect of the selector to be examined separately from the effect of the prediction model. It also means that the selected-comment totals should not be multiplied by four and described as four independent datasets.

4.5.2 BERT Features and Prediction Models

BERT provides contextual representations for the selected comments. The stored maximum token length is 256, while each resulting comment representation has 768 dimensions. The sparse branch averages comment representations into an author-level vector. The LSTM branch retains an ordered sequence and accounts for the valid sequence lengths when processing padded batches. These features apply the contextual representation

approach described by Devlin et al. (2019), while the comparison in this study concerns the downstream author prediction methods.

The sparse branch uses ElasticNet in the reported configuration, combining regularization terms when fitting the continuous targets. The saved lasso condition names are retained in the artifact folder for identification, but should be expanded as Lasso/ElasticNet or sparse regression in the report. This prevents a reader from assuming that all results were produced by pure Lasso regularization. Regularized regression remains relevant because the number of embedding features is large compared with the number of training authors (Friedman et al., 2010).

The sequence branch uses a stacked bidirectional LSTM with two recurrent layers, 128 hidden units in each direction and dropout of 0.2. The recorded training configuration allows 35 epochs with a batch size of four and a learning rate of 0.001. Its final layer produces five continuous values, one for each Big Five trait. Low/High decisions are obtained afterwards by applying the saved thresholds. The LSTM is therefore trained as a continuous predictor, even though the application also reports classification measures derived from its outputs.

[Insert Code Listing 4.1 here: Short extract showing the two model representations and continuous outputs. Use approximately 12–20 relevant lines from the feature construction and LSTM output code. Identify the pooled 768-dimensional author vector, the comment sequence and the five-value output. Omit unrelated imports, complete classes and lengthy training loops.]

4.5.3 Training-Only GAN Augmentation

The augmentation component generates paired embedding and OCEAN-score samples. Its purpose is to expand the information available during fitting, particularly where trait bands have limited representation. The saved configuration uses a latent dimension of 64, a hidden dimension of 128, 150 GAN epochs and a synthetic-sample weight of 0.35. Generated examples remain training material and are not counted as additional human authors in the validation or test sets.

The saved augmentation report marks all four GAN conditions as used. Each sparse GAN condition generated 50 rows, while each LSTM GAN condition generated 50

sequences. The baseline sequence augmentation was fitted from 8,675 training timesteps, compared with 5,926 timesteps for the Q-learning sequence condition. The targeted training pool contains 50 author rows in each reported condition. Thus, the label targeted augmentation should not be taken to mean that the GAN was fitted on only a small subset of the 50 training authors in this run.

The generated sequences are assembled from the embedding-and-score generation procedure. They should not be interpreted as newly collected author histories or automatically assumed to preserve realistic changes in an individual's writing over time. The adversarial framework provides a way of learning a synthetic distribution, but its success in producing useful training observations requires evaluation on real held-out data (Goodfellow et al., 2014). The next sections therefore discuss the measured effect of augmentation rather than treating an increased training-set size as an improvement by itself.

4.6 System Testing and Verification

4.6.1 Evidence Available for Testing

System testing concerns whether the application performs the intended operations, while model evaluation concerns the quality of its predictions. Both are needed in a research application. A completed training run does not prove that every interface control handles invalid input correctly. In the same way, a functioning form and a correctly saved database row do not prove that a prediction is accurate. The testing discussion therefore separates recorded experiment checks from interface tests that still require documented execution.

The current run provides metrics and saved model states for all eight conditions. Its supporting reports record the author partitions, comment counts, selection statistics and augmentation use. These files are evidence that the experimental workflow produced outputs for the reported conditions. The author-overlap check also reports no repeated author identifiers across the three sampled partitions. These observations support the implementation of the main experimental stages.

The research-contract report contains 80 checks, comprising 79 passes, one warning and no failures. The warning concerns reproducibility: a repository commit identifier was recorded, but the working directory contained changes, so the commit alone does not fully identify the code used. This result should be described as a warning rather than an entirely

clean reproducibility check. Keeping the actual code snapshot together with the configuration would provide stronger support for repeating the run.

[Insert Table 4.3 here: System testing and verification record. Use columns for test case, input or action, expected outcome, observed outcome, evidence and status. Include valid experiment submission, invalid settings, unauthenticated access, run-status update, result retrieval, author separation, saved condition files and prediction-profile retrieval. Mark interface cases as pending until executed and recorded. For existing artifact checks, report the observed counts and the reproducibility warning rather than marking every row as passed.]

4.6.2 Limits of the Saved Classification Audit

The classification audit file is marked PASS and records 200 metric checks and 40 threshold checks. However, the detailed rows reviewed contain null values for recomputed measures and confusion counts. The separate prediction-evidence file contains no usable author prediction rows. As a result, the PASS label alone does not demonstrate an independent reconstruction of every classification measure from the underlying predictions.

This distinction became important when the sparse and LSTM result fields were compared. The code and saved reports show that some similarly named classification fields describe different calculations. A test that merely confirms the presence of a field can succeed without establishing that two fields have the same meaning. A stronger verification step would retain the observed score, predicted score, observed binary label and predicted binary label for each test author and trait, and then recalculate the reported measures from those rows.

The current chapter therefore reports the existence and limits of the checks honestly. It does not assign success to unexecuted interface tests or present unavailable confusion counts as measured results. Completing the testing table and the common binary evaluation will strengthen the final report without changing the basic scope of the project. The required work is verification of the existing implementation rather than the introduction of an additional prediction method.

4.7 Continuous Prediction Results

4.7.1 Basis of the Regression Comparison

Continuous prediction is the primary task because the observed OCEAN values are numerical scores. Mean Absolute Error (MAE) describes the average absolute difference between the predicted and observed values. Root Mean Squared Error (RMSE) gives greater weight to larger errors. Lower values of either measure indicate smaller prediction errors. The coefficient of determination, R^2 , and Pearson correlation provide additional views of how the predictions relate to the observed variation.

The overall figures in the saved comparison are means across the five traits. In particular, the reported RMSE is an average of the trait-specific RMSE values, rather than a newly calculated square root over every author-trait error combined. The interpretation uses this stored averaging method consistently. The regression measures are on the normalized target scale and should not be described as classification accuracy percentages.

[Insert Table 4.4 here: Held-out regression results for all eight conditions. Populate from comparison.csv and confirm each entry against the test section of the condition metrics. Columns should include condition, selection method, GAN setting, MAE, RMSE, R^2 and Pearson correlation. Use consistent rounding to four decimal places. State the run identifier, 10 test authors and averaging over five traits below the table.]

4.7.2 Results of the Sparse Regression Conditions

The sparse baseline recorded an MAE of 0.23076 and an RMSE of 0.26656. Adding Q-learning without GAN augmentation changed these values to 0.21934 and 0.26548. The corresponding reduction in MAE was 0.01142 on the normalized scale. This is a favourable change for that matched comparison, although the reduction in RMSE was much smaller. The result suggests that selecting fewer comments did not necessarily remove information needed for this particular sparse model.

Adding GAN augmentation to the sparse baseline produced an MAE of 0.22054 and an RMSE of 0.26416. Compared with the unaugmented baseline, the MAE decreased by 0.01022. However, combining Q-learning and GAN augmentation produced an MAE of 0.22838 and an RMSE of 0.26672. The combined condition had a larger MAE than either the

sparse Q-learning condition without GAN or the sparse baseline with GAN. The effects of the two additions were therefore not simply cumulative.

These results show why each matched condition is needed. If the analysis only compared the original baseline with the fully combined pipeline, it would overlook the different behaviour of Q-learning and GAN when applied separately. The sparse results provide evidence of small changes in error within this sample, but do not establish that the same changes would occur with a different set of training authors or a different random seed.

4.7.3 Results of the LSTM Conditions

The LSTM baseline recorded an MAE of 0.23032 and an RMSE of 0.26252. With Q-learning and no GAN, the MAE changed to 0.22916, while the RMSE increased to 0.26764. This combination illustrates that a reduction in average absolute error can occur alongside an increase in the measure that gives greater weight to larger errors. It would therefore be incomplete to discuss only the small improvement in MAE.

The baseline LSTM with GAN recorded an MAE of 0.22652 and an RMSE of 0.26136. The LSTM condition with both Q-learning and GAN recorded an MAE of 0.22960 and an RMSE of 0.26142. Compared with the unaugmented Q-learning LSTM, augmentation slightly increased MAE by 0.00044 but reduced RMSE by 0.00622. The two error measures again describe different aspects of the prediction behaviour.

The LSTM results demonstrate that adding a sequence model does not automatically produce a large improvement over pooled regression in this sample. The model has access to the order and variation of comment embeddings, but it is fitted using only 50 real training authors. The saved run shows modest differences between conditions. No claim of a statistically established advantage is made because this chapter does not contain repeated-run estimates or a significance analysis of those differences.

[Insert Figure 4.3 here: Comparison of test MAE and RMSE. Use two simple panels with the same eight condition labels as Table 4.4. Show lower values as better and keep the condition order consistent across panels. Avoid a single combined score or a highlighted overall winner. Source: comparison.csv from the reported run.]

4.7.4 R², Correlation and Trait-Level Findings

The mean R² values are negative for all eight conditions, ranging from approximately −0.1499 to −0.0659. This is an important limitation of the continuous results. A negative R² for a trait means that its squared prediction error exceeds that of predicting the observed mean of that test trait. The negative overall averages therefore do not support a claim of strong explained variation across the traits. They also do not mean that every individual trait in every condition has a negative value.

For example, the baseline LSTM records an Openness R² of 0.0312, while its mean R² across the five traits is −0.07126. Its trait MAE values are 0.1377 for Openness, 0.2701 for Conscientiousness, 0.1930 for Extraversion, 0.2448 for Agreeableness and 0.3060 for Neuroticism. This shows that the overall mean hides meaningful differences between trait results. A single average should therefore be supported by a trait-level table.

The recorded mean Pearson correlations also vary, from approximately −0.0609 to 0.2050 across conditions. Correlation describes whether predicted scores tend to rise and fall with observed scores; it does not directly describe the size of prediction error. A condition can have a relatively favourable correlation and still produce scores that are poorly calibrated. Both error and association measures are needed before interpreting a model as useful for individual author estimates.

[Insert Table 4.5 here: Trait-level regression results. Present the five OCEAN traits and their MAE and R² for the eight conditions, using two compact panels if necessary. Include the full results rather than only the trait with the strongest value. Source: the test/per_trait blocks in the condition metrics files.]

4.8 Threshold Selection and Classification Results

4.8.1 Validation-Based Decision Thresholds

The prediction models produce continuous scores before the classification step. A candidate decision threshold divides those scores into predicted Low and High labels. The threshold search is carried out on validation predictions. For each candidate, the system calculates F1 and specificity and combines them using the selection score: $2 \times \text{F1} \times \text{specificity} \div (\text{F1} + \text{specificity})$. These inputs are measured proportions between zero and one, not fixed constants chosen in advance.

The selected threshold is the candidate with the strongest recorded selection score, with additional measures used to resolve ties. The candidate values are obtained from the distribution of predicted scores. When there are more than 19 valid prediction scores, percentile-based candidates are used; with 19 or fewer scores, the distinct observed predictions supply the candidates. In this run, the validation set contains ten authors, so the latter procedure applies. Candidate values are rounded and restricted to the normalized range before evaluation. Thus, the final model threshold is selected by a validation metric search rather than taken directly from the training median or fixed at 0.5 for every trait.

The selected threshold is then retained when evaluating the test authors. For illustration, the baseline LSTM records decision thresholds of 0.7551, 0.4865, 0.3479, 0.3911 and 0.4667 for O, C, E, A and N respectively. These are specific to this condition and run. They should not be described as universal personality boundaries, and they should not be substituted for the observed-label medians listed in Section 4.4.3.

[Insert Table 4.6 here: Validation-selected decision thresholds for all eight conditions and five traits. Obtain the values from each condition's selected_threshold fields and verify that the source is validation. Add a separate note listing the five training-median observed-label cutoffs. Do not place these two types of boundary under the same column heading.]

[Insert Code Listing 4.2 here: Threshold-selection logic. Include only the candidate evaluation, harmonic F1-specificity score, tie handling and selection of the validation threshold. Approximately 12–20 lines are sufficient. Add a short note that the retained threshold is applied unchanged to test predictions.]

4.8.2 Interpretation of Threshold Graphs

A threshold graph shows how classification measures change as the decision boundary moves. It does not show a new training run at every point. The predicted continuous scores are held fixed while different boundaries produce different Low/High decisions. Increasing the threshold generally makes a High prediction harder to obtain, which can reduce false positives while also missing some genuinely High observations.

The graph used to explain threshold selection should be drawn from validation data and mark the retained validation threshold. A test threshold sweep may be shown as an additional description of behaviour, but its highest point should not be used to replace the

validation-selected boundary. Selecting a new threshold from the test graph would use the final evaluation data for a model decision and weaken the interpretation of the held-out result.

The current presentation plots may display a reduced set of candidate positions rather than every numerical threshold. A horizontal label such as threshold class 1 to 5 can therefore represent an ordered display position, not five personality classes and not one common threshold shared by all traits. For the final report, using the actual numerical threshold on each trait's horizontal axis provides a clearer explanation and avoids this ambiguity.

[Insert Figure 4.4 here: Validation threshold sensitivity for a clearly named condition. Use five small OCEAN panels showing F1 and specificity against the actual threshold value. Mark the retained threshold with a vertical line. Use validation rows from threshold_sweeps_long.csv; avoid replacing the final threshold with a peak observed on the test split.]

4.8.3 Recorded Classification Findings and Reporting Issue

The baseline LSTM records mean test accuracy of 0.6000, mean High-class F1 of 0.5811 and mean specificity of 0.5133. Its trait accuracies are 0.9 for Openness, 0.5 for Conscientiousness, 0.5 for Extraversion, 0.5 for Agreeableness and 0.6 for Neuroticism. These figures show variation between traits. The average accuracy does not mean that 60 per cent of authors had all five traits classified correctly.

The saved baseline LSTM validation accuracy is 0.6400 and its validation F1 is 0.6581. The corresponding test values are lower. This difference is consistent with the need to report a separate test evaluation after decisions have been made on validation data. With only ten authors in each held-out split, however, these values remain sensitive to the composition of the sample. One changed decision affects an individual trait's accuracy by ten percentage points.

A reporting inconsistency prevents a direct combined accuracy ranking across the two model families. In the sparse branch, the saved accuracy and macro-F1 fields are calculated from three trait bands. Its F1, precision, recall and specificity fields separately describe binary threshold decisions. In the LSTM branch, the accuracy and F1 fields used in the main

comparison describe binary Low/High decisions. The saved comparison therefore places values with different meanings beside one another.

For this reason, the sparse baseline accuracy of 0.36 should not be compared directly with the LSTM baseline accuracy of 0.60 as proof of a binary classification advantage. Similarly, a field named macro-F1 should not automatically be interpreted as a two-class macro average. The common classification table remains pending until the same observed binary labels, retained thresholds and metric definitions are used for every condition. This reporting issue does not change the recorded regression errors, but it limits the classification conclusions that can be drawn from the current summary.

[Insert Table 4.7 here after verification: Common binary test results for all eight conditions. Recalculate accuracy, High-class F1, precision, recall and specificity from retained author predictions using the training-median labels and validation-selected thresholds. Average the same measures over the five traits. Do not copy the mixed accuracy column directly from comparison.csv. Preserve the verified author-level evidence used to obtain the table.]

[Insert Figure 4.5 here after verification: OCEAN confusion matrices for the condition selected by the researcher. Show actual counts of true Low, false High, false Low and true High predictions, with ten authors per trait. The current prediction-evidence export is empty, so these counts must be reconstructed and verified before plotting.]

4.9 Effect of Comment Selection and GAN Augmentation

4.9.1 Comment Reduction and Processing Time

The baseline selection retained all 11,019 available comments, whereas Q-learning retained 7,598 comments. This is a reduction of 3,421 comments, or approximately 31.05 per cent of the full available comment input. Mean selected comments per author decreased from 157.41 to 108.54. These figures provide direct evidence that the implemented selector reduced the amount of text represented in the selected feature set.

The saved report describes this reduction as BERT embedding calls saved. In interpreting that field, it is more precise to describe comments avoided at the embedding-input stage. Batching and reuse of cached embeddings can change the number of actual model executions. The comment reduction is therefore a useful workload measure, but is not itself a measured percentage reduction in wall-clock time or energy consumption.

The recorded feature-building time was approximately 3.23 seconds for the baseline and 28.81 seconds for Q-learning. In this run, fewer selected comments did not correspond to a shorter recorded feature-building stage. The selector introduces additional processing, and cache use may affect the relative timings. Without a controlled comparison of cache state and timing boundaries, it would be misleading to claim that the 31.05 per cent comment reduction produced an equal speed improvement.

The condition training times also differed considerably. The sparse baseline and sparse Q-learning conditions took approximately 12.89 and 13.65 seconds respectively. The unaugmented LSTM baseline and Q-learning conditions took approximately 591.96 and 256.48 seconds. These observations show that the relationship between comment selection and elapsed time depends on the later model stage. They remain timings from one recorded run rather than hardware-independent performance guarantees.

[Insert Figure 4.6 here: Selection volume and recorded processing time. Use separate panels for total selected comments and condition training time. Include the feature-building times in the caption or a small accompanying note. A logarithmic time axis may be used if clearly labelled because the augmented baseline LSTM is much slower than the other conditions. Do not combine comments and seconds on an unexplained shared scale.]

4.9.2 Matched Effects of the Additional Components

Without GAN augmentation, Q-learning reduced sparse MAE by 0.01142 and LSTM MAE by 0.00116. With GAN augmentation, switching from baseline selection to Q-learning increased sparse MAE by 0.00784 and LSTM MAE by 0.00308. These matched comparisons indicate that the effect of comment selection depends on the augmentation setting. The average MAE change across all four Q-learning comparisons is a small reduction of 0.000415, which should not obscure the different directions of the individual results.

GAN augmentation reduced MAE in the baseline-selection conditions by 0.01022 for sparse regression and 0.00380 for LSTM. Under Q-learning, however, it increased MAE by 0.00904 for sparse regression and 0.00044 for LSTM. The average change across the four GAN comparisons is a reduction of 0.001135. This average is descriptive; it does not establish that adding GAN consistently helps or that its effect is statistically reliable.

The cost of augmentation also varies by condition. Recorded training time was approximately 186.04 seconds for the sparse baseline with GAN and 50.59 seconds for sparse Q-learning with GAN. The augmented baseline LSTM recorded 56,253.64 seconds, or about 15.63 hours, while the augmented Q-learning LSTM recorded 883.46 seconds, or about 14.72 minutes. The very large baseline LSTM value should be checked against execution logs before it is treated as a repeatable benchmark. The saved duration alone cannot identify the contribution of sequence length, resource contention or interruption.

These observations support a practical interpretation of the component comparisons. Reducing input volume can be useful even where an accuracy gain is not demonstrated, and a small error reduction may come with a substantial training cost. The final choice therefore needs to state which objective is most important. It should also recognise that combining two individually useful components does not guarantee that their combined condition will perform better.

4.10 Discussion in Relation to the Project Objectives

4.10.1 Data Preparation, Selection and Representation

The first objective concerned the preparation of labelled social media comments with separate author-level training, validation and test data. The saved sample and overlap reports provide evidence of that separation for the 70 authors used in this experiment. The reports also identify the source files and normalized score scale. The result supports the implemented preparation workflow, while the small sample and limited record of earlier cleaning remain relevant to the strength of the findings.

The second objective concerned Q-learning selection before BERT and comparison with a baseline. Both conditions were implemented and measured. The selected-comment total decreased by approximately 31.05 per cent, but the changes in prediction error were mixed across augmentation settings. The objective of carrying out a controlled selection comparison was therefore addressed. The stronger claim that Q-learning consistently improves prediction or total processing speed is not established by the present evidence.

The third objective concerned the extraction of 768-dimensional contextual embeddings and the construction of pooled features and ordered sequences. The implementation provides both representations, allowing the sparse and LSTM branches to

operate on the same underlying author sample. The results demonstrate that the two branches can be fitted and evaluated within a common experiment. Since no alternative text representation is evaluated here, the study does not isolate the benefit of BERT over every possible feature-extraction method.

4.10.2 Augmentation, Model Comparison and Evaluation

The fourth objective concerned paired embedding-and-score augmentation and comparison with unaugmented training. The saved reports confirm generation in all four GAN conditions. The resulting held-out errors differ across the matched comparisons, providing evidence for discussing when the added stage helped or harmed the recorded performance. Generated observations remain derived training material, so their presence should not be used to enlarge the reported number of independent participants.

The fifth objective concerned comparison of sparse regression and the stacked bidirectional LSTM. Both model families produced continuous predictions and saved results under the four matched selection-and-augmentation settings. The error measures reveal different trade-offs, and all eight conditions have negative mean R^2 in the reported run. These findings support a cautious comparison of the implemented approaches rather than an assumption that the more complex model must be more suitable.

The sixth objective concerned reporting regression, classification, threshold and efficiency measures on held-out data. Regression results, selected thresholds and comment-efficiency reports are available. However, the common classification comparison still requires correction of the metric meanings and preservation of the underlying prediction evidence. The objective has therefore been addressed in the implementation, with a specific reporting limitation that must be resolved before the final classification table and model verdict are completed.

[Insert Table 4.8 here: Relationship between objectives and findings. Use the six objectives from Chapter One, the implemented component, evidence available and the remaining limitation. Keep the table concise and distinguish a completed implementation from a demonstrated performance improvement.]

4.10.3 Expected Outcomes and Observed Findings

The expected outcome was an integrated research system capable of estimating OCEAN scores and comparing the proposed processing choices. The saved artifacts support the completion of the eight-condition workflow. The experiment also provides evidence that learned selection can reduce comment input and that augmentation changes prediction behaviour. These are useful implementation and research outcomes even though the resulting errors do not demonstrate strong predictive performance.

The expectation that sequence processing would preserve information lost through averaging remains a reason for including LSTM, rather than a result that can be assumed. Similarly, the expectation that synthetic samples would assist underrepresented trait regions requires support from the actual held-out measurements. In this run, the mixed matched effects and negative mean R^2 show why expected benefits must be tested. The findings should therefore be reported as observed rather than adjusted to match the original expectations.

The study contributes a structured comparison and a means of inspecting its evidence. Its current findings are most useful for understanding how these components behave together on the sampled PANDORA records. They do not establish clinical usefulness, suitability for recruitment decisions or general performance across Nigerian university students. No such target-population evaluation was carried out in the reported experiment.

4.11 Limitations Affecting the Interpretation of Results

The main limitation is the number of independent authors. Fifty real training observations provide limited support for a model with many parameters, even when each author supplies several comments. The validation and test sets each contain ten authors, making classification measures particularly sensitive to individual cases. Additional comments and synthetic training examples cannot be counted as additional independent test participants.

The chapter reports one saved run with seed 42. Differences between conditions may reflect the particular sampled authors and the behaviour of stochastic training. No confidence intervals or claims of statistical significance are presented because the required repeated-run evidence is not included. If later experiments are used to strengthen the report, their results

should be identified separately rather than silently substituted into tables belonging to this run.

The classification reporting issue limits comparison between the sparse and sequence branches. The audit status does not remove that limitation because the available detailed evidence is insufficient for independent reconstruction. The final binary table and confusion matrices remain marked for completion. The recorded regression comparison can still be discussed, but the negative mean R^2 values restrict claims about the quality of the continuous predictions.

Reproducibility is also limited by the recorded changes in the working directory and the incomplete runtime specification. A commit identifier, configuration file and random seed are helpful, but do not fully identify the executed system when unrecorded code changes remain. Timing results require similar care because cache state, available resources and background activity were not fully controlled in the evidence reviewed.

Finally, the application display and the experimental pipeline are related but distinct sources of evidence. A readable profile page confirms a presentation feature; it does not validate an inferred personality score. The outputs remain research estimates derived from English-language online comments and supplied dataset labels. Their interpretation should remain within this scope, as already stated in Chapter One.

4.12 Selection of the Preferred Model Pipeline

The final selection is reserved for the researcher. The preceding sections provide the evidence needed to compare the conditions without assigning a single overall winner. The decision should state whether priority is given to continuous-score error, verified Low/High classification, reduced comment processing or acceptable training cost. Where more than one consideration is used, the reason for the chosen balance should be explained in plain terms.

The recorded automatic findings file applies its own ranking rules, but those labels are not adopted as the project's final verdict here. In particular, the combined classification comparison must first use consistent definitions. The chosen condition should be identified by its exact model, selection and augmentation settings, and the decision should acknowledge the sample size and observed limitations. If the intended use is continuous personality estimation, the regression results should remain central to that explanation.

[Researcher to complete: The preferred pipeline for this project is _____. It combines _____ prediction with _____ comment selection and _____ augmentation. It was selected because _____, based on the verified results in Tables 4.4–4.7 and the processing evidence in Figure 4.6. Its main trade-off is _____. This choice is limited to the experimental scope described in this chapter.]

4.13 Chapter Summary

This chapter has presented the implementation outcomes and the findings from the selected PANDORA experiment. The application brings together author-level data preparation, comment selection, BERT representation, training-only augmentation and two continuous prediction approaches. The eight conditions were evaluated using the same sampled author partitions, allowing the effects of the main processing choices to be examined through matched comparisons.

The results show reduced comment input under Q-learning, mixed changes in prediction error when selection and augmentation are added, and considerable differences in recorded training time. Negative mean R^2 values and the small held-out sample limit claims of strong predictive performance. The review also identified a classification reporting inconsistency and incomplete prediction evidence, which are marked for verification before the final classification presentation is completed. The preferred pipeline remains open for the researcher's decision, while the next chapter can draw conclusions and recommendations from the verified findings.

REFERENCES

- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies* (Vol. 1, pp. 4171–4186). Association for Computational Linguistics.
- Friedman, J., Hastie, T., & Tibshirani, R. (2010). Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1), 1–22.
- Goodfellow, I., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative adversarial nets. In *Advances in Neural Information Processing Systems* (Vol. 27, pp. 2672–2680).
- Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.